# Wireless Intrusion Detection System for Wi-Fi Deauthentication Attacks

A machine-learning prototype using AWID3 wireless traffic

Repository: https://github.com/kakeetopius/wids


## Abstract

This project develops an offline wireless intrusion detection system (WIDS) for identifying Wi-Fi deauthentication traffic. Two Random Forest models are evaluated on the same AWID3 holdout. The primary behavioural model excludes frame type and subtype so that it must use timing, radio and traffic-rate information. A protocol-feature model includes those fields as a comparison baseline. The behavioural model achieves 98.14% accuracy and 96.51% deauthentication recall. The baseline achieves 100% on this dataset, but its result is more likely to depend on AWID3-specific fields. The behavioural model is therefore used for dashboard alerts.


## 1. Aim and system scope

Deauthentication flooding can repeatedly disconnect wireless clients. The aim was to build a reproducible prototype that labels traffic as normal or malicious, reports accuracy and a confusion matrix, and raises deauthentication alerts. The system is a WIDS, not a WIPS: it does not transmit frames, capture a live network, block devices or change network configuration.

## 2. Data and features

The experiment uses the AWID3 deauthentication capture files. The raw dataset is not included because of its size; the trained artifact is included so the dashboard can run without it. Both models use frame length, timing, radio information, duration, frame flags, reason code, signal strength and one-second frame-rate features for all traffic, sources and destinations.

The behavioural model excludes `wlan.fc.type` and `wlan.fc.subtype`. These are protocol fields strongly associated with the label in this subset. The second model includes them deliberately to show how much they affect the result. This comparison is important: the perfect baseline score is not treated as proof of perfect real-world detection.

## 3. Evaluation method

Complete capture files, rather than individual random rows, define the split. This keeps frames from one capture together and reduces leakage. Files 25–28 are used only for validation and threshold selection. Files 29–32 are held back for the final test. All other available files are used for training. Both models use the same Random Forest settings, median imputation and fixed random seed.

| Partition | Files | Purpose |
|---|---|---|
| Training | Outside 25–32 | Fit both models |
| Validation | Deauth_25 to Deauth_28 | Select alert threshold |
| Final test | Deauth_29 to Deauth_32 | Report performance |

## 4. Results

The final test contains 33,869 observations: 15,844 normal and 18,025 deauthentication. The behavioural threshold is 0.35; the protocol baseline threshold is 0.25.

| Model | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|
| Behavioural detector | 98.14% | 99.99% | 96.51% | 98.22% |
| Protocol baseline | 100.00% | 100.00% | 100.00% | 100.00% |

**Behavioural model confusion matrix:**

| Actual / predicted | Normal | Deauth |
|---|---|---|
| **Normal** | 15,843 | 1 |
| **Deauth** | 629 | 17,396 |

The behavioural model misses 629 attack-labelled observations but produces only one false alarm. The protocol baseline makes no errors on this particular holdout. Its perfect result is explained by the highly informative frame fields and should be tested on independent captures before being treated as a general detector.

## 5. Implementation and demonstration

The training pipeline is in `src/wids/train.py`. It stores both models and their metrics in `models/deauth_model.joblib`. The FastAPI dashboard loads the behavioural model for offline replay alerts and displays the model comparison on the Model page. The health endpoint reports that live capture and automatic response are disabled.

To verify the prototype, install with `uv sync`, start `uv run uvicorn wids.web.app:app --host 127.0.0.1 --port 8000`, then inspect the Model page, Detections page and `/api/health`. These pages demonstrate the stored evaluation; they are not live wireless monitoring.

## 6. Limitations and future work

The results come from one AWID3 subset and one attack class. The protocol baseline is especially dataset-dependent, while the behavioural model is a more conservative estimate but can still miss attacks. Future work should test both models on independent authorised captures, measure performance across environments, calibrate probabilities, add other attack classes and consider live monitor-mode ingestion only on an isolated approved testbed.

## 7. Conclusion

The project meets the WIDS requirements by detecting deauthentication traffic, raising alerts and reporting measurable results. Presenting the behavioural detector alongside the protocol baseline makes the result more transparent: the primary model is strong but imperfect, while the perfect score is clearly identified as a dataset-specific comparison.

## AI assistance

AI coding assistants supported repository inspection, code refactoring and dashboard integration. The project author wrote the documentation and reviewed the implementation, metrics, model choice and safety boundary.